CO 2 – ASSESSMENT TOOL -2

COURSE CODE: **DSA0308**

COURSE NAME: **NLP**

Reg no:**192425108** Name: **K.Sadiq hussain**

Q1. Morphological processing: analyzing, analysis, analytical

Aim: Identify roots and affixes, classify inflectional and derivational forms, and normalize related words for search engine indexing.

Python program:

```
words = ["analyzing", "analysis", "analytical"]


morphology = {
   "analyzing": {
      "root": "analyze",
      "prefix": "None",
      "suffix": "-ing",
      "type": "Inflectional",
      "normalized": "analyze"
   },
   "analysis": {
      "root": "analy",
      "prefix": "None",
      "suffix": "-sis",
      "type": "Derivational (irregular form)",
      "normalized": "analyze"
   },
   "analytical": {
      "root": "analysis",
      "prefix": "None",
```

```
        "suffix": "-ical",
        "type": "Derivational",
        "normalized": "analyze"
    }
}

print(f"{'Word':<15}{'Root':<15}{'Suffix':<12}"
      f"{'Type':<30}{'Normalized'}")

for word in words:
    item = morphology[word]
    print(f"{word:<15}{item['root']:<15}"
          f"{item['suffix']:<12}{item['type']:<30}"
          f"{item['normalized']}")
```

OUTPUT:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/Co2 at2 q1.py
Word           Root           Suffix      Type                           Normaliz
ed
analyzing      analyze        -ing        Inflectional                   analyze
analysis       analy          -sis        Derivational (irregular form)  analyze
analytical     analysis       -ical       Derivational                   analyze
```

Q2. Morphological parsing: disagree, agreement, agreeable

Aim: Identify prefixes, roots, suffixes, derivational types, and semantic changes for sentiment analysis.

Python program:

```
words = ["disagree", "agreement", "agreeable"]

morphology = {
    "disagree": {
        "prefix": "dis-",
```

```
        "root": "agree",
        "suffix": "None",
        "type": "Prefix derivation",
        "meaning": "To have a different or opposing opinion",
        "normalized": "agree"
    },
    "agreement": {
        "prefix": "None",
        "root": "agree",
        "suffix": "-ment",
        "type": "Derivational",
        "meaning": "The state or result of agreeing",
        "normalized": "agree"
    },
    "agreeable": {
        "prefix": "None",
        "root": "agree",
        "suffix": "-able",
        "type": "Derivational",
        "meaning": "Pleasant or acceptable",
        "normalized": "agree"
    }
}

print(f"{'Word':<15}{'Prefix':<10}{'Root':<10}"
      f"{'Suffix':<10}{'Type':<22}{'Normalized'}")

for word in words:
    item = morphology[word]
    print(f"{word:<15}{item['prefix']:<10}"
          f"{item['root']:<10}{item['suffix']:<10}"
          f"{item['type']:<22}{item['normalized']}")
```

```
    print("Meaning:", item["meaning"])
```

OUTPUT:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q2.py
Word           Prefix    Root      Suffix    Type                  Normalized
disagree       dis-      agree     None      Prefix derivation     agree
Meaning: To have a different or opposing opinion
agreement      None      agree     -ment     Derivational          agree
Meaning: The state or result of agreeing
agreeable      None      agree     -able     Derivational          agree
Meaning: Pleasant or acceptable
>>>
```

Q3. Morphology-based normalization: govern, government, governance

Aim: Identify roots, derivational suffixes, and derivational levels, then normalize the words for topic modeling and document clustering.

Python program:

```
words = ["govern", "government", "governance"]


morphology = {
    "govern": {
        "root": "govern",
        "affixes": "None",
        "level": "Level 0 - Base",
        "normalized": "govern"
    },
    "government": {
        "root": "govern",
        "affixes": "-ment",
        "level": "Level 1 - Derivation",
        "normalized": "govern"
    },
    "governance": {
```

```
        "root": "govern",
        "affixes": "-ance",
        "level": "Level 1 - Derivation",
        "normalized": "govern"
    }
}

print(f"{'Word':<15}{'Root':<12}{'Affix':<12}"
      f"{'Derivational Level':<28}{'Normalized'}")

for word in words:
    item = morphology[word]
    print(f"{word:<15}{item['root']:<12}"
          f"{item['affixes']:<12}{item['level']:<28}"
          f"{item['normalized']}")
```

OUTPUT:

```
>>>
    = RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q3.py
    Word           Root        Affix       Derivational Level          Normalized
    govern         govern      None        Level 0 - Base              govern
    government     govern      -ment       Level 1 - Derivation        govern
    governance     govern      -ance       Level 1 - Derivation        govern
>>>
```

Q4. Morphological parsing: activate, activation, reactivation

Aim: Identify prefixes, roots, suffixes, derivational sequences, and changes in word class and meaning.

Python program:

```
words = ["activate", "activation", "reactivation"]

morphology = {
    "activate": {
        "prefix": "None",
```

```
        "root": "active",
        "suffix": "-ate",
        "sequence": "active -> activate",
        "word_class": "Adjective -> Verb",
        "meaning": "To make active",
        "normalized": "activate"
    },
    "activation": {
        "prefix": "None",
        "root": "active",
        "suffix": "-ate, -ion",
        "sequence": "active -> activate -> activation",
        "word_class": "Adjective -> Verb -> Noun",
        "meaning": "The process of making active",
        "normalized": "activate"
    },
    "reactivation": {
        "prefix": "re-",
        "root": "active",
        "suffix": "-ate, -ion",
        "sequence": "active -> activate -> reactivate -> reactivation",
        "word_class": "Adjective -> Verb -> Noun",
        "meaning": "The process of activating again",
        "normalized": "activate"
    }
}

print(f"{'Word':<16}{'Prefix':<10}{'Root':<10}"
      f"{'Suffix':<16}{'Normalized'}")

for word in words:
    item = morphology[word]
```

```
    print(f"{word:<16}{item['prefix']:<10}"
          f"{item['root']:<10}{item['suffix']:<16}"
          f"{item['normalized']}")
    print("Sequence:", item["sequence"])
    print("Word class:", item["word_class"])
    print("Meaning:", item["meaning"])
    print("-" * 70)
```

OUTPUT:

```
>>>
= RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q4.py
Word            Prefix    Root      Suffix          Normalized
activate        None      active    -ate            activate
Sequence: active -> activate
Word class: Adjective -> Verb
Meaning: To make active
----------------------------------------------------------------------
activation      None      active    -ate, -ion      activate
Sequence: active -> activate -> activation
Word class: Adjective -> Verb -> Noun
Meaning: The process of making active
----------------------------------------------------------------------
reactivation    re-       active    -ate, -ion      activate
Sequence: active -> activate -> reactivate -> reactivation
Word class: Adjective -> Verb -> Noun
Meaning: The process of activating again
----------------------------------------------------------------------
```

Q5. Inflectional normalization: create, creates, creating

Aim: Identify grammatical suffixes, extract root forms, classify tense and aspect-related forms, and normalize words for search optimization.

Python program:

```
words = ["create", "creates", "creating"]


morphology = {
    "create": {
        "suffix": "None",
        "category": "Base form",
        "root": "create",
```

```
        "normalized": "create"
    },
    "creates": {
        "suffix": "-s",
        "category": "Third-person singular present",
        "root": "create",
        "normalized": "create"
    },
    "creating": {
        "suffix": "-ing",
        "category": "Present participle",
        "root": "create",
        "normalized": "create"
    }
}

print(f"{'Word':<15}{'Suffix':<10}{'Category':<36}"
      f"{'Root':<12}{'Normalized'}")

for word in words:
    item = morphology[word]
    print(f"{word:<15}{item['suffix']:<10}"
          f"{item['category']:<36}{item['root']:<12}"
          f"{item['normalized']}")
```

OUTPUT:

```
>>>
    = RESTART: C:/Users/kakar/AppData/Local/Programs/Python/Python314/CO2 AT2 Q5.py
    Word           Suffix    Category                            Root        Normali
    zed
    create         None      Base form                           create      create
    creates        -s        Third-person singular present       create      create
    creating       -ing      Present participle                  create      create
```